

Распознавание лиц на C++ CPU и CUDA

Краткое техническое руководство по активному коду Hive: одиночный запуск, batch 50 и batch 500, реальные границы CPU/GPU и измеренное ускорение.

Фактический путь одного запроса



CPU: декодирование, подготовка, разбор YuNet, alignment и решение | GPU: YuNet/SFace через ONNX Runtime CUDA и собственный score kernel

Что именно доказывает демонстрация

Обе версии выполняют одинаковую задачу YuNet + SFace по одним изображениям и эталонам. CPU-версия запускает нейросети строго по одному изображению. CUDA-версия передаёт весь B-image batch в нативные ONNX Runtime CUDA sessions и затем вычисляет матрицу сходства B x N собственным kernel из sface_cuda.cu.

Python не выполняет нейросеть. Он остаётся транспортом HTTP/Ursina. Активные вычислительные процессы - identity_cpu.exe и identity_cuda.exe, собранные из C++ и CUDA C++.

Где реально работает CPU, а где GPU

Этап	Устройство	Что делает код
Загрузка изображения	CPU / C++	image_io.cpp декодирует JPEG/PNG в RGBA.
Подготовка YuNet	CPU / C++	prepare_detection_batch создаёт NCHW BGR тензор и padding.
YuNet inference	GPU в CUDA-версии	Ort::Session использует CUDAExecutionProvider для dynamic batch B.
Разбор детекций	CPU / C++	Код читает 12 выходов YuNet, выбирает лицо и пять landmarks.
Alignment 112 x 112	CPU / C++	align_faces вычисляет similarity transform и bilinear sampling.
SFace inference	GPU в CUDA-версии	Второй Ort::Session создаёт 128D embedding для всего batch.
Нормализация	CPU / C++	Каждый 128D вектор нормализуется до длины 1.
Cosine scores	GPU / CUDA C++	sface_cuda.cu запускает B x N независимых оценок.
Порог и JSON	CPU / C++	Выбираются имя, runner-up, score, margin и accepted.

Важно: текущий код не утверждает, что абсолютно каждый шаг выполняется на GPU. Ускоряются самые тяжёлые нейросетевые графы YuNet/SFace и массовое сравнение embeddings; подготовка, alignment и финальное решение остаются нативным C++ на CPU.

Один и тот же код для 1, 50 и 500

Batch	Фактический запуск	Смысл
1	Один путь через CUDA provider	Проверяет правильность и минимальную задержку; массовый параллелизм ещё не заполнен.
50	Один dynamic batch B=50	YuNet/SFace получают 50 элементов за один запрос, затем kernel считает 50 x N оценок.
500	Один dynamic batch B=500	Если используются 50 кадров x10, все 500 позиций проходят нейросети; повторы не пропускаются.

Что означает параллельность CUDA

Batch 500 не означает 500 отдельных программ и не гарантирует, что все 500 изображений физически исполняются в один такт. ONNX Runtime разбивает свёртки и матричные операции на CUDA kernels, а GPU распределяет тысячи threads по доступным SM. В собственном score kernel запускается двумерный grid: B строк запросов x ceil(N/256) блоков эталонов, по одному score на допустимый thread.

Повтор одного изображения десять раз остаётся десятью заданиями. benchmark_native.ps1 формирует B путей, worker возвращает ровно B результатов, а тест падает при несовпадении count или expected identity.

Активные файлы и ключевые команды

Файл	Команда	Точное назначение
sface_engine.cpp	options.AppendExecutionProvider_CUDA	Подключает CUDAExecutionProvider к YuNet и SFace sessions.
sface_engine.cpp	yunet.Run(...) / sface.Run(...)	Запускает два ONNX-графа для полного dynamic batch.
sface_engine.cpp	for (...) embed_images({images[index]})	Создаёт намеренно последовательную CPU-базу: один кадр за один вызов.
sface_cuda.cu	cudaMalloc / cudaMemcpy H2D	Выделяет device-буферы и загружает Bx128 queries и Nx128 references.
sface_cuda.cu	cosine_scores_kernel<<<grid, block>>>	Запускает собственный kernel: block=256, grid=(ceil(N/256), B).
sface_cuda.cu	dot += query[d] * reference[d]	Один thread выполняет 128 multiply-add для одной пары query/reference.
sface_cuda.cu	cudaMemcpy D2H / cudaFree	Возвращает BxN scores и освобождает все три GPU-буфера.
sface_identity_cli.cpp	--serve-tsv	Держит модели и эталоны загруженными между запросами Hive.

Проверенное сравнение на этом компьютере

Intel Core i9-14900K + NVIDIA GeForce RTX 4060. Нативные Release binaries, одинаковые модели, изображения и эталоны, предварительный прогрев каждой формы batch.

Нагрузка	C++ CPU последовательно	C++/CUDA batch	Ускорение	Корректность
50	6551,4 мс	2840,1 мс	2,31x	50/50
500	65304,2 мс	9365,7 мс	6,97x	500/500

Старые цифры Python/PyTorch из предыдущего PDF удалены. Таблица выше относится только к текущим identity_cpu.exe и identity_cuda.exe. Для batch 500 CUDA сокращает время примерно с 65,3 до 9,37 секунды.

Почему преимущество растёт с batch

Фактор	Объяснение
CPU baseline	Каждый кадр отдельно проходит YuNet, alignment и SFace; следующая итерация начинается после предыдущей.
CUDA batch	Один большой тензор лучше заполняет GPU, а стоимость запуска kernels и runtime распределяется на большее число изображений.
Score kernel	Все $B \times N$ пары независимы; CUDA назначает отдельный thread каждой допустимой паре.
Ограничение	Передачи памяти, host alignment и малые batch уменьшают выигрыш; GPU не обязан быть быстрее для одиночного кадра.

Как воспроизвести

1	Собрать Release: <code>.\setup_windows.ps1 -CudnnBin PATH_TO_CUDNN_BIN</code>
2	Проверить наличие <code>identity_cpu.exe</code> , <code>identity_cuda.exe</code> , ONNX Runtime CUDA DLL, CUDA runtime и cuDNN 9.
3	Запустить одинаковый тест: <code>.\benchmark_native.ps1 -RuntimePaths PATH_TO_CUDNN_BIN</code>
4	Открыть <code>results/native_face_cpu_cuda_results.csv</code> и проверить <code>batch</code> , <code>correct</code> , <code>errors</code> , <code>recognition_ms</code> и <code>images_per_second</code> .
5	Для Hive оставить <code>AI_MIPS_NATIVE_FACE=1</code> ; <code>server.py</code> запускает нативный persistent worker и не выполняет inference в Python.

Итог для защиты проекта

Проект показывает контролируемое сравнение: последовательный C++ CPU против пакетного C++/CUDA на той же задаче. Результат не основан на пропуске повторов или кэше ответов. На RTX 4060 преимущество составляет 2,31x для 50 и 6,97x для 500 распознаваний. Главная причина - пакетное выполнение YuNet/SFace на CUDA и независимое параллельное вычисление $B \times N$ cosine scores.